Kevin Küster

Zusammenfassung: DB_KK

Datenbanken

SQL - Select

```
SELECT [DISTINCT] spalten
FROM tabellen
[WHERE bedingung]
[GROUP BY spalten]
[HAVING gruppen-bedingung]
[ORDER BY spalten ASC/DESC]
```

Beispiel

```
-- Kunden aus München mit
-- Bestellsumme > 100:
SELECT k.name, SUM(b.betrag) AS gesamt
FROM kunde k
JOIN bestellung b ON k.kNr = b.kNr
WHERE k.stadt = 'München'
GROUP BY k.kNr, k.name
HAVING SUM(b.betrag) > 100
ORDER BY gesamt DESC
```

=> WHERE filtert Zeilen vor Grouping - keine Aggregate
HAVING filtert Gruppen nach Grouping - Aggregate erlaubt

DDL - Struktur definieren

DDL - Data Definition Language

```
-- Create Table mit wichtigsten Constraints:
CREATE TABLE bestellung(
  bestellNr INTEGER NOT NULL,
  kundenNr INTEGER NOT NULL,
  datum DATE DEFAULT CURRENT_DATE,
  betrag DECIMAL(10,2) CHECK (betrag > 0),
  PRIMARY KEY (bestellNr),
  FOREIGN KEY (kundenNr)
    REFERENCES kunde(kundenNr)
    ON DELETE CASCADE
```

CONSTRAINT	BEDEUTUNG
PRIMARY KEY	Eindeutig + NOT NULL
FOREIGN KEY	Verweis auf PK anderer Tabelle
NOT NULL	Kein NULL erlaubt
UNIQUE	Eindeutig (NULL erlaubt)
CHECK (B)	B darf nicht FALSE sein
DEFAULT x	Standardwert bei fehlendem Wert
ON DELETE CASCADE	Löscht abhängige Zeilen mit
——"—— RESTRICT	Löschen verweigern (Default)

DML - Daten einfügen/ändern/löschen

DML - Data Manipulation Language

```
-- Insert: neue Zeile einfügen
INSERT INTO kunde(kundenNr, name, stadt)
VALUES (42, 'Müller', 'München');

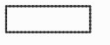
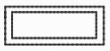
-- UPDATE: bestehende Zeile verändern
UPDATE kunde
SET stadt = 'Berlin'
    name = 'Müller-Berlin'
WHERE kundenNr = 42;

-- DELETE: Zeilen löschen
DELETE FROM kunde
where kundenNr = 42;
=> UPDATE/DELETE ohne
   Where -> alle Zeilen
```

ACID	Bedeutung
A (Atomicity)	Ganz oder gar nicht
C (Consistency)	Konsistenter Anfang-/Endzustand
I (Isolation)	Keine gegenseitige Beeinflussung
D (Durability)	Nach Commit dauerhaft gespeichert

ER - Diagramme

- Entity: Repräsentiert reales Objekt, von anderen unterscheidbar durch Attribute
- Attribut: Eigenschaft einer Entity, ordnet Wert aus Wertebereich zu
- Primärschlüssel: Minimale Attributmenge, die Entities eines Typs eindeutig identifizieren
- Entity-Typ: Menge gleichartiger Entities
- Relationship (Beziehung): Zusammenhang zwischen zwei oder mehr Entities
- Beziehungstyp: Menge gleichartiger Beziehungen


Entity-Typ,

Schwacher (abhängiger Entity-Typ)


Beziehungstyp,

Beziehung zu schwachen ET


Attribut, Unterstrichen - Schlüsselattribut

- Kardinalitäten:

Zweistellige Beziehungstypen klassifiziert nach Kardinalität:

- 1:1-Beziehung: Jede Entity aus E1 steht höchstens zu einer Entity aus E2 in Beziehung und umgekehrt
  Angestellter <-1:1-> Personalakte
- 1:n-Beziehung: Jede Entity aus E2 steht höchstens zu einer Entity aus E1, aber eine Entity aus E1 kann mit mehreren aus E2 in Beziehung stehen
  Firma <-1:n-> Angestellter
- m:n-Beziehung: Entities aus E1 können mit mehreren aus E2 in Beziehung stehen und umgekehrt
  Aktionär <-m:n-> Firma

Relationale Algebra -> SQL

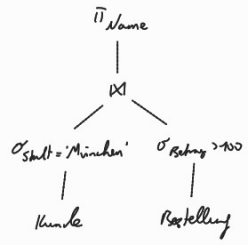
σ (Selektion)	$\sigma_B(R)$	WHERE B
π (Projektion)	$\pi_L(R)$	SELECT DISTINCT L
β (Umbenennung)	$\beta_{(neu \leftarrow alt)}$	AS neu
$\bowtie_\theta$ (Join)	$R \bowtie_\theta S$	JOIN ... ON θ
$*\bowtie$ (Left-outer-join)	$R *\bowtie S$	LEFT OUTER JOIN
$\cup$ (Vereinigung)	$R \cup S$	UNION
- (Differenz)	$R \setminus$	EXCEPT

Beispiel

```
-- Namen Münchener Kunden mit Bestellung > 100:
```

$\pi_{Name}(\sigma_{stadt='München'}(Kunde) \bowtie \sigma_{Betrag>100}(Bestellung))$

Operatorbaum:



=> Optimierter Operatorbaum:
Selektionen/Projektionen so weit wie möglich nach unten

Integritätsregeln

↳ Teil des Datenbankmodells

PK-Integrität	Jeder PK-Wert ist eindeutig (Keine Duplikate)
Entity-Integrität	PK-Wert ist niemals NULL
Referentielle Integrität	FK ist NULL oder verweist auf existierenden PK
Mengeeigenschaft	Keine identisch gleichen Tupel in einer Relation

---

Normalformen (NF) 1, 2, 3

▷ 1. NF:

↳ Alle Attribute atomar (keine Mengen, keine zusammengesetzten Werte)

↳ Jede Zelle enthält genau einen Wert

↳ Maßnahme: Zusammengesetzte Attribute aufspalten, Wiederholungsgruppen in eigene Relation

▷ 2. NF:

↳ 1NF erfüllt

↳ Jedes Nichtschlüsselattribut ist voll funktional abhängig vom gesamten Primärschlüssel

⇒ Relevant nur bei zusammengesetzten Schlüssel

↳ Maßnahme: Attribute die nur von einem Teil des Schlüssels abhängen; ⇒ eigene Relation

▷ 3NF:

↳ 2 NF erfüllt

↳ Kein Nichtschlüsselattribut ist transitiv abhängig von Primärschlüssel

⇒ Keine Abhängigkeit: Schlüssel → X → Y

↳ Maßnahme: Transitive Abhängigkeiten in eigene Tabelle auslagern

Zerlegung in 3. NF

(i) Alle funktionalen Abhängigkeiten (FDs) sammeln

(ii) Schlüssel bestimmen

(iii) Für jede FD $X \to Y$, die 3NF verletzt:

⇒ Neue Relation $R(X,Y)$ mit X als Schlüssel

(iv) Prüfen ob der Schlüssel der Originalrelation noch in einer Teilrelation enthalten ist (sonst hinzufügen)

(v) Redundante Relationen entfernen

Beispiel

$R(A,B,C,D,E,F,G)$; FDs: $A \to BCDE$, $D \to EF$, $F \to G$; PK: $\{A\}$

Verletzung: $A \to D \to EF$, $A \to D \to E \to \dots$, $D \to F \to G$

⇒ $R1(A,B,C,D)$; $R2(D,E,F)$, $R3(F,G)$